Introduction

In the ever-evolving world of software development, Kubernetes has emerged as a pivotal tool in the orchestration of containerized applications. It simplifies deployment, scaling, and operations, offering a robust platform for managing containerized services at scale. This guide is crafted for system administrators, developers, and DevOps professionals who are looking to harness the power of Kubernetes in a stable and reliable environment.

Our journey will take us through the process of setting up a Kubernetes (K8s) cluster on Ubuntu 22.04 LTS. Ubuntu's long-term support (LTS) version ensures stability and extended support, making it an ideal choice for hosting our Kubernetes cluster. Whether you're setting up a cluster for production, development, or learning purposes, this guide aims to provide a clear and comprehensive path forward.

We'll start by understanding the prerequisites, including the hardware and software requirements, and a brief overview of the necessary technical background. Following that, we'll dive into the practical steps of configuring the master and worker nodes, detailing each command and its purpose. The guide also includes troubleshooting tips and verification steps to ensure your cluster is up and running smoothly.

By the end of this article, you will have a fundamental Kubernetes cluster up and running, ready to be the backbone of your containerized applications. Let's embark on this journey to unlock the potential of Kubernetes on Ubuntu 22.04 LTS.

Prerequisites

Ubuntu 22.04 LTS

Familiarity with Linux, CLI, and Kubernetes concepts

sudo privileges on your machines

Setting up the Master Node

First, we'll set up the master node, which will manage the cluster.

Step 1 : Update and Upgrade Ubuntu

Begin by updating your package lists and upgrading the existing packages to their latest versions.

sudo apt-get update

sudo apt-get upgrade -y

Step 2 : Disable Swap

sudo swapoff -a

sudo sed -i '/ swap / s/^\(.*\)$/#\1/g' /etc/fstab

Step 3 : Add Kernel Parameters

sudo tee /etc/modules-load.d/containerd.conf <<EOF

overlay

br_netfilter

EOF

sudo modprobe overlay

sudo modprobe br_netfilter

sudo tee /etc/sysctl.d/kubernetes.conf <<EOF

net.bridge.bridge-nf-call-ip6tables = 1

net.bridge.bridge-nf-call-iptables = 1

net.ipv4.ip_forward = 1

EOF

Reload the change

sudo sysctl --system

Step 4 : Install Containerd Runtime

sudo apt install -y curl gnupg2 software-properties-common apt-transport-https ca-certificates

sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmour -o /etc/apt/trusted.gpg.d/docker.gpg

sudo add-apt-repository "deb [arch=amd64] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable"

sudo apt update

sudo apt install -y containerd.io

containerd config default | sudo tee /etc/containerd/config.toml >/dev/null 2>&1

sudo sed -i 's/SystemdCgroup \= false/SystemdCgroup \= true/g' /etc/containerd/config.toml

sudo systemctl restart containerd

sudo systemctl enable containerd

Step 5 : Install Kubernetes Components:

Add the Kubernetes signing key and repository.

sudo apt-get update

apt-transport-https may be a dummy package; if so, you can skip that package

sudo apt-get install -y apt-transport-https ca-certificates curl gpg

curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.29/deb/Release.key | sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg

This overwrites any existing configuration in /etc/apt/sources.list.d/kubernetes.list

echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.29/deb/ /' | sudo tee /etc/apt/sources.list.d/kubernetes.list

Step 6 : Update the package list and install kubelet, kubeadm, and kubectl.

sudo apt-get update

sudo apt-get install -y kubelet kubeadm kubectl

sudo apt-mark hold kubelet kubeadm kubectl

Step 7 : Initialize Kubernetes Master Node: Initialize the Kubernetes cluster using kubeadm.

sudo kubeadm init

Output:

[init] Using Kubernetes version: v1.29.0

[preflight] Running pre-flight checks

[preflight] Pulling images required for setting up a Kubernetes cluster

[preflight] This might take a minute or two, depending on the speed of your internet connection

[preflight] You can also perform this action in beforehand using 'kubeadm config images pull'

W1217 13:50:54.561046 43242 checks.go:835] detected that the sandbox image "registry.k8s.io/pause:3.6" of the container runtime is inconsistent with that used by kubeadm. It is recommended that using "registry.k8s.io/pause:3.9" as the CRI sandbox image.

[certs] Using certificateDir folder "/etc/kubernetes/pki"

[certs] Generating "ca" certificate and key

[certs] Generating "apiserver" certificate and key

[certs] apiserver serving cert is signed for DNS names [fbb-k8s-master kubernetes kubernetes.default

kubernetes.default.svc kubernetes.default.svc.cluster.local] and IPs [10.96.0.1 10.201.65.50]

[certs] Generating "apiserver-kubelet-client" certificate and key

[certs] Generating "front-proxy-ca" certificate and key

[certs] Generating "front-proxy-client" certificate and key

[certs] Generating "etcd/ca" certificate and key

[certs] Generating "etcd/server" certificate and key

[certs] etcd/server serving cert is signed for DNS names [fbb-k8s-master localhost] and IPs [10.201.65.50 127.0.0.1 ::1]

[certs] Generating "etcd/peer" certificate and key

[certs] etcd/peer serving cert is signed for DNS names [fbb-k8s-master localhost] and IPs [10.201.65.50 127.0.0.1 ::1]

[certs] Generating "etcd/healthcheck-client" certificate and key

[certs] Generating "apiserver-etcd-client" certificate and key

[certs] Generating "sa" key and public key

[kubeconfig] Using kubeconfig folder "/etc/kubernetes"

[kubeconfig] Writing "admin.conf" kubeconfig file

[kubeconfig] Writing "super-admin.conf" kubeconfig file

[kubeconfig] Writing "kubelet.conf" kubeconfig file

[kubeconfig] Writing "controller-manager.conf" kubeconfig file

[kubeconfig] Writing "scheduler.conf" kubeconfig file

[etcd] Creating static Pod manifest for local etcd in "/etc/kubernetes/manifests"

[control-plane] Using manifest folder "/etc/kubernetes/manifests"

[control-plane] Creating static Pod manifest for "kube-apiserver"

[control-plane] Creating static Pod manifest for "kube-controller-manager"

[control-plane] Creating static Pod manifest for "kube-scheduler"

[kubelet-start] Writing kubelet environment file with flags to file "/var/lib/kubelet/kubeadm-flags.env"

[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/config.yaml"

[kubelet-start] Starting the kubelet

[wait-control-plane] Waiting for the kubelet to boot up the control plane as static Pods from directory "/etc/kubernetes/manifests". This can take up to 4m0s

[apiclient] All control plane components are healthy after 8.001522 seconds

[upload-config] Storing the configuration used in ConfigMap "kubeadm-config" in the "kube-system" Namespace

[kubelet] Creating a ConfigMap "kubelet-config" in namespace kube-system with the configuration for the kubelets in the cluster

[upload-certs] Skipping phase. Please see --upload-certs

[mark-control-plane] Marking the node fbb-k8s-master as control-plane by adding the labels: [node-role.kubernetes.io/control-plane node.kubernetes.io/exclude-from-external-load-balancers]

[mark-control-plane] Marking the node fbb-k8s-master as control-plane by adding the taints [node-role.kubernetes.io/control-plane:NoSchedule]

[bootstrap-token] Using token: qyq7z7.5auk8bcovin856fj

[bootstrap-token] Configuring bootstrap tokens, cluster-info ConfigMap, RBAC Roles

[bootstrap-token] Configured RBAC rules to allow Node Bootstrap tokens to get nodes

[bootstrap-token] Configured RBAC rules to allow Node Bootstrap tokens to post CSRs in order for nodes to get long term certificate credentials

[bootstrap-token] Configured RBAC rules to allow the csrapprover controller automatically approve CSRs from a Node Bootstrap Token

[bootstrap-token] Configured RBAC rules to allow certificate rotation for all node client certificates in the cluster

[bootstrap-token] Creating the "cluster-info" ConfigMap in the "kube-public" namespace

[kubelet-finalize] Updating "/etc/kubernetes/kubelet.conf" to point to a rotatable kubelet client certificate and key

[addons] Applied essential addon: CoreDNS

[addons] Applied essential addon: kube-proxy

Your Kubernetes control-plane has initialized successfully!

To start using your cluster, you need to run the following as a regular user:

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Alternatively, if you are the root user, you can run:

```
export KUBECONFIG=/etc/kubernetes/admin.conf
```

You should now deploy a pod network to the cluster.

Run "kubectl apply -f [podnetwork].yaml" with one of the options listed at:

https://kubernetes.io/docs/concepts/cluster-administration/addons/

Then you can join any number of worker nodes by running the following on each as root:

```
kubeadm join 10.10.10.1:6443 --token qyq7z7.5augt56vin856fj \
 --discovery-token-ca-cert-hash
sha256:09a1fc88c9fc8cab5171333546456455dd54cf3d8cb3400362586185308c3
```

To start using your cluster, set up the kubeconfig.

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
```

sudo chown $(id -u):$(id -g) $HOME/.kube/config

Step 8 : Deploy a Pod Network: Install a pod network so that your nodes can communicate with each other.

kubectl apply -f
https://raw.githubusercontent.com/projectcalico/calico/v3.25.0/manifests/calico.yaml

Setting up the Worker Nodes

Now, configure your worker nodes.

1. Proceed above step 1– 6 on the worker nodes

2. Join the Worker Nodes:

On each worker node, use the join command you got from the master node's init output.

**kubeadm join 10.10.10.1:6443 --token qyq7z7.5augt56vin856fj **

--discovery-token-ca-cert-hash
sha256:09a1fc88c9fc8cab5171333546456455dd54cf3d8cb3400362586185308c3

Verifying the Cluster Setup

Ensure your cluster is up and running.

kubectl get nodes

Output:

```
NAME              STATUS   ROLES          AGE     VERSION
fbb-k8s-master    Ready    control-plane  10m     v1.29.0
fbb-k8s-worker-1  Ready    <none>         2m31s   v1.29.0
```

Conclusion

Congratulations! You've successfully set up a Kubernetes cluster on Ubuntu 22.04 LTS. This basic cluster is ready for deploying applications and further exploration of Kubernetes' capabilities.

```
sudo apt-get update
sudo apt-get upgrade -y
sudo swapoff -a
sudo sed -i '/ swap / s/^\(.*\)$/#\1/g' /etc/fstab
sudo tee /etc/modules-load.d/containerd.conf <<EOF
overlay
br_netfilter
EOF
sudo modprobe overlay
sudo modprobe br_netfilter
sudo tee /etc/sysctl.d/kubernetes.conf <<EOF
net.bridge.bridge-nf-call-ip6tables = 1
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1
EOF
```

```
sudo sysctl --system

sudo apt install -y curl gnupg2 software-properties-common apt-transport-https ca-certificates

sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmour -o /etc/apt/trusted.gpg.d/docker.gpg

sudo add-apt-repository "deb [arch=amd64] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable"

sudo apt update

sudo apt install -y containerd.io

containerd config default | sudo tee /etc/containerd/config.toml >/dev/null 2>&1

sudo sed -i 's/SystemdCgroup \= false/SystemdCgroup \= true/g' /etc/containerd/config.toml

sudo systemctl restart containerd

sudo systemctl enable containerd

sudo apt-get update

sudo apt-get install -y apt-transport-https ca-certificates curl gpg

curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.29/deb/Release.key | sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg

echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.29/deb/ /' | sudo tee /etc/apt/sources.list.d/kubernetes.list

sudo apt-get update

sudo apt-get install -y kubelet kubeadm kubectl

sudo apt-mark hold kubelet kubeadm kubectl
```